



25

API Design & REST (FastAPI)

US3.2 · C7

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Concevoir une API de prédiction (endpoints, schémas, erreurs)
- › Charger le modèle au démarrage ; distinguer /health et /ready
- › Valider les I/O avec Pydantic (contrat du module 22)
- › Relier API, run MLflow et Model Card à trois niveaux

Déroulé de la séance

- 1 **Théorie les concepts clés**
- 2 **Travaux dirigés** TP guidé, correction au fil de l'eau
- 3 **Autonomie tutorée** vous avancez, formateur dispo
- 4 **Bilan synthèse + preuves**

PREUVE FINALE contrat 200/401/422 · Model Card métier/maintenance/AI Act traçable

1

Module 25 — les concepts clés

Théorie

■ Scénario atelier & enjeux

La supervision de l'atelier veut interroger ton modèle depuis son propre logiciel. Tu ne vas pas leur envoyer un notebook : tu exposes une URL qui répond en JSON.

Angle éthique / risque (C2) Une API ouverte sans contrat expose le modèle à des usages détournés. Définir le schéma (Pydantic) borne ce qui entre et ce qui sort.

■ Contexte & outils — pourquoi ce module

Contexte : un modèle est inutilisable s'il reste dans un script. Pourquoi : on l'EXPOSE en service que d'autres systèmes peuvent appeler.

- FastAPI : framework d'API web rapide, avec doc /docs automatique
- Pydantic : valide et type les entrées/sorties (contrat I/O)
- Uvicorn : le serveur qui fait tourner l'API
- REST / HTTP : verbes + codes de retour (200, 401, 422, 503)
- /health : sonde de vie pour savoir si le service répond

Objectif du module Un endpoint /predict-tabular qui répond proprement, documenté et testé.

■ Une API, c'est un contrat

Une API expose le modèle via des endpoints au comportement prévisible : entrées, sorties, erreurs.



i

Le jargon décodé

REST = style d'API sur HTTP. Endpoint = une URL+verbe. Pydantic valide et documente. Swagger/OpenAPI = doc auto (/docs).

?

Le saviez-vous ?

FastAPI génère tout seul la doc interactive /docs depuis tes schémas : la doc ne peut pas mentir, elle dérive du code.

EN UNE PHRASE Codes HTTP : 200 ok · 401 auth · 422 validation · 503 pas prêt.

■ /health vs /ready : vivant n'est pas prêt

/health dit « le process tourne » (liveness) ; /ready dit « le modèle est chargé » (readiness).

/health — liveness

- le process tourne
- toujours 200 si l'API est up

/ready — readiness

- le modèle est chargé
- 503 tant qu'aucun modèle

! À retenir

Le modèle se charge UNE fois au démarrage (lifespan), jamais à chaque requête (sinon latence catastrophique).

? Idée reçue

« /health et /ready, c'est pareil. » FAUX : un service peut être vivant mais pas prêt (modèle en cours de chargement).

EN UNE PHRASE L'orchestrateur ne route le trafic que vers les instances prêtes.

■ Codes HTTP & traçabilité du modèle

Request-id pour la requête ; version et Model Card pour le modèle servi.

- 401 (auth) ≠ 422 (validation) ≠ 503 (indispo) : choisir le code juste est un acte de design
- model_version + run_id MLflow réel — sinon la mention explicite « à produire »



Astuce de pro

Un client envoie M-7 ? Normalise au bord (M-7 → MACH-07) : on accepte l'hétérogène sans polluer la logique interne.



Cas réel InduSense

Sans clé → 401 ; historique trop court → 422 ; modèle absent → 503 ; requête correcte → 200 + proba_panne.

EN UNE PHRASE Requête suivie par request-id ; modèle suivi par sa Model Card.

■ Le contrat de l'API, en vrai (squelette)

Quatre routes, un contrat porté par Pydantic, des codes HTTP qui disent ce qui s'est passé.

```
GET /health -> 200 {"status":"ok"} # liveness
GET /ready -> 200 {model_version} | 503 # readiness
POST /predict-tabular -> 200 {proba_panne, decision, ...}
# X-API-Key requis (sinon 401) ; readings: min 7 (sinon 422)
class TabularPredictionRequest(BaseModel):
    machine_id: str
    readings: list[SensorReading] = Field(..., min_length=7)
```

→ Cas réel InduSense

require_api_key -> 401 si clé absente OU invalide (pas 422). Le middleware request-id ajoute X-Request-ID à chaque requête.

? Le saviez-vous ?

FastAPI génère /docs (Swagger) depuis ces schémas : la doc est dérivée du code, elle ne peut pas mentir.

EN UNE PHRASE Un contrat testé (200/401/422/503), c'est la preuve de la compétence C7.

■ À retenir en 3 points

- Un endpoint est une porte d'action : /predict, /health.
- Pydantic valide les entrées et rejette ce qui ne respecte pas le contrat.
- /docs documente l'API automatiquement.

2

Module 25 — TP guidé, correction au fil de l'eau

Travaux dirigés

■ TP — endpoints & schémas

```
# requête type (payload.json)
POST /predict-tabular + header X-API-Key
{ "machine_id":"MACH-07", "readings":[ {timestamp,temperature,pressure_bar} ] }
```

Normalisation au bord Un client peut envoyer M-7 / MACH_07 → normalize_machine_id côté API.

■ TP — tester avec TestClient

```
uv run pytest tests/test_api.py -q
# acquis : /health 200 · sans clé 401 · historique court 422 · prédiction 200
# /ready 503 : non acquis tant que le test dédié n'existe pas
```

✓ Preuve attendue /docs · /health 200 · prédiction 200 · 401 sans clé · 422 historique court.

3

Module 25 — vous avancez, le formateur reste disponible

Autonomie tutorée

■ Model Card — trois niveaux, preuves réelles

Produisez docs/model_card.md, puis validez la preuve attendue.

- Métier — finalité, décision assistée, limites et supervision humaine
- Technique / maintenance — version, données, métriques, seuil, coût, run_id réel ou « à produire »



Conformité AI Act & benchmark distinct

Usage, risques et traçabilité ; classe « à confirmer ». Marine ≠ apprenant : XGB 24 h · seuil $\approx 0,41$ · PR-AUC $\approx 0,62$ · prévalence 16,6 % ; coût frugal 0,158 vs lourd 0,352 gCO₂e.

■ Definition of Done & transition

- DoD : API testée 200/401/422 + Model Card trois niveaux sans preuve inventée

Transition → Module 26 Le service est exposé : on modélise les menaces et on le durcit.